ASSIGNMENT 5 - CO5

PART A - DESIGN

TASK A4 - EVALUATE CONTAINERISATION

4 Marks | Bloom's Level: K5 - Evaluate

Case Study - University Result Portal

The assignment describes a university result portal implemented as a Python web application and a database. It serves a few hundred users on normal days, but 150,000 students log in within two hours on result day. The current portal runs on one physical Linux server and crashes during the peak. The university has a small private server room and an account with a public cloud provider.

Containerisation can package the portal and its dependencies into repeatable units, isolate process views and limit resource consumption. It can make application replicas quicker to deploy than full guest operating systems. However, containers share the host Linux kernel; containerisation alone does not add physical capacity or eliminate the need for a resilient multi-host design.

1. A4.1 - How Containers Work (2 Marks)

1.1 Linux namespaces

A Linux namespace wraps a global system resource so that processes in a container see an isolated view of that resource. A container runtime creates and configures namespaces when it starts a container. Namespaces provide separation of process IDs, network interfaces, mount points and other kernel-visible resources; they are isolation of views, not separate guest kernels. [1]

Namespace	What it isolates / virtualises	Relevance to the portal
PID	Process ID space and process visibility.	The portal container sees its own process tree; processes on the host or in another container are not visible through its PID namespace view.
NET	Network devices, addresses, routes, firewall-related network state and socket ports.	Each service can have its own network stack; controlled ports connect Nginx, application and database tiers.
MNT	Mount table and filesystem mount points.	A container receives its own filesystem view; selected host paths or volumes are mounted deliberately.
UTS	Hostname and NIS domain-name view.	Give each portal replica a container-specific hostname.
IPC	System V IPC and POSIX message-queue resources.	Isolates inter-process communication objects between services.
USER	User and group ID mappings and privilege view.	Container root can be mapped to an unprivileged host identity with user namespaces/rootless operation.
CGROUP	View of cgroup hierarchy and membership paths.	Can restrict what cgroup details a container can see; resource control itself is performed by cgroup controllers.

Namespaces answer “what can this process see and access?” For example, a PID namespace separates process IDs and a network namespace gives the container its own network stack. A mount namespace gives it a separate view of mount points. This helps run multiple services on one host without exposing every host resource to every service. [1]

1.2 Control groups (cgroups)

Control groups organise processes hierarchically and account for or limit their use of host resources. In cgroup v2, controllers can manage resources such as CPU, memory, I/O and process count. This complements namespaces: namespaces isolate the view of resources, while cgroups govern how much of selected resources a workload may consume. [2]

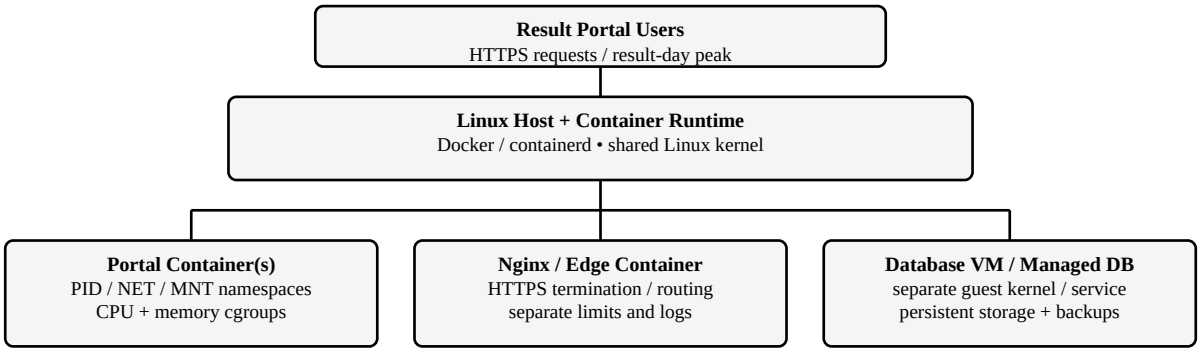
Controller / control	Purpose	Portal application
CPU	CPU time weights and quota limits.	Prevent a runaway web-worker container from consuming all host CPU during result-day load.
Memory	Memory accounting, protection and maximum limits; reclaim/OOM behaviour is controller-dependent.	Set a memory ceiling for each application replica and alert before it reaches the limit.

Controller / control	Purpose	Portal application
I/O	Weight or bandwidth controls for supported block I/O devices.	Prevent a busy logging or application process from starving database storage I/O.
PIDs	Limit the number of processes/tasks in a cgroup.	Reduce the risk of fork/process exhaustion.

For the result portal, allocate CPU and memory limits based on load tests, then monitor actual usage. Limits should not be set so low that the service is throttled during the expected peak. Reserve host capacity for the kernel, container runtime, Nginx and essential monitoring. Cgroups control resources but are not a complete security boundary. [1][2]

1.3 How a container image is built in layers

A container image is a read-only template composed of filesystem layers. A base image supplies user-space files and runtime components; later filesystem-changing build instructions add application dependencies and source files. Identical layers can be reused by other images, reducing repeated storage and download work. When a container is created, its image layers are stacked and a container-specific writable layer is added on top. [3]



Recommended hybrid: scale stateless app containers; isolate persistent database separately.
A container shares the host kernel; a VM runs a full guest OS and its own kernel.

Layer / storage area	Example content	Behaviour
Base layer	Minimal Linux user-space and Python runtime.	Read-only; can be shared by images that use the same base.
Dependency layer	Installed Python packages from requirements.	Reusable when dependency inputs do not change.
Application layer	Portal source code and static assets.	Read-only part of the built image.
Container writable layer	Runtime temporary files and changes made while the container runs.	Unique per container; discarded when that container is removed unless data is stored elsewhere.

A Dockerfile organises these steps. Placing dependency installation before frequently changed source code can preserve the dependency layer in the build cache. The image history can be inspected with `docker history`. Database files, uploaded results and other important state should be placed in persistent volumes or a managed storage service rather than the disposable writable layer. [3][4]

2. A4.2 - Containers versus Virtual Machines (2 Marks)

Comparison	Containers	Virtual machines
Architecture	Isolated user-space processes sharing the host OS kernel.	Each VM runs a complete guest OS, including its own kernel, above a hypervisor.
Start-up	Usually seconds because no separate guest OS has to boot; image availability and application initialisation still matter.	Usually minutes or longer because the guest OS must boot and initialise services; templates and snapshots can shorten some workflows.
Memory	Smaller per instance: application process, required libraries and container runtime overhead; kernel is shared.	Higher per instance: guest kernel, OS services, application and hypervisor/device overhead.
Isolation	Namespaces isolate resource views; cgroups control resources. Containers share the host kernel.	Separate guest kernels and hypervisor boundary provide stronger isolation for many threat models.
Security	Needs kernel/runtime hardening: non-root user, least privilege, dropped capabilities, seccomp/AppArmor, read-only filesystems and restricted daemon access.	Stronger boundary against many guest-to-guest OS-level threats, but hypervisor flaws, management-plane mistakes and guest vulnerabilities remain possible.

Containers and VMs are complementary, not mutually exclusive. A VM can host a Linux kernel and container runtime, with containers running inside that VM. The comparison is therefore about the isolation boundary and

resource model, not simply “secure” versus “insecure.” [5]

2.1 Decision for the university portal

Use containers for the stateless Python web/application tier. Build one versioned image and run multiple replicas behind Nginx or a load balancer. Apply CPU and memory limits, run as a non-root user, avoid privileged mode, drop unnecessary capabilities and monitor cgroup usage. This allows replicas to start in seconds and scale out when result-day traffic increases.

Use a managed database service or a dedicated database VM for the primary persistent database. This keeps durable data, database maintenance and backup/restore policy separate from the lifecycle of application containers and provides a stronger isolation boundary. If the university must containerise the database, use persistent volumes, non-root execution, network restrictions, replication and tested backup/restore; never store the only database copy in the container writable layer.

For the small private server room, run the container host on a supported Linux server or a VM cluster. The public cloud account can provide an additional application replica tier or managed database/recovery service. The portal's stateless tier can scale horizontally, but the database must be sized for connection count, transaction rate and durable I/O. A container restart cannot correct an undersized database or single-host failure by itself.

3. Alternative Concept - Portal and Database in Separate VMs

If both components ran in VMs, create one application VM and one database VM, each with its own guest OS and kernel. The application VM would host Nginx/Gunicorn or the Python application; the database VM would host the database engine and its persistent virtual disk. Place the VMs on separate hosts or use host anti-affinity where available, restrict application-to-database network traffic and back up the database independently.

Evaluation item	Container design	One VM per component
Start-up time	Normally seconds for a container, subject to image pull, runtime and app initialisation.	Typically minutes for a full guest OS boot and service initialisation; snapshots/templates can reduce provisioning work but do not remove guest OS startup.
Memory per instance	Shares one host kernel; memory mainly covers application/runtime plus container overhead.	Each VM needs guest OS kernel, services, application/database memory and hypervisor overhead.
Isolation strength	Process and resource isolation on a shared kernel; kernel vulnerabilities can affect multiple containers.	Separate guest kernels and hypervisor boundary generally provide stronger separation between application and database.
Operations	One host kernel/runtime to patch, but each image/runtime still needs updates and security control.	Two guest operating systems to patch, monitor, back up and administer; clearer OS-level separation but greater resource use.

3.1 Start-up and memory evaluation

The question explicitly contrasts VM start-up measured in minutes with container start-up measured in seconds. This is a typical architectural comparison, not a guarantee for every platform: VM templates, warm pools, guest boot configuration, container image pulls and application health checks affect observed time. The important reason is that a VM boots a full guest OS, whereas a container starts an isolated process using the already-running host kernel. [5]

Let M_g be the memory required by a guest OS and its services, M_a the application memory, and M_c container/runtime overhead. Two separate VMs require approximately $M_{host} + (M_g + M_{app}) + (M_g + M_{db})$ plus hypervisor/cache reserve. A container deployment requires $M_{host} + M_c + M_{app} + M_{db}$ plus shared-kernel/cache reserve. Exact savings depend on guest distribution, workload and memory configuration; no fixed MB/GB value can be claimed from the assignment data.

3.2 Isolation and security evaluation

Separate VMs give the application and database independent guest kernels. A compromise of one guest is generally less likely to directly expose the other guest's process space or kernel than in a shared-kernel container design. This is why a separate database VM can be attractive for sensitive student results. VMs still require a patched hypervisor, secure management access, network segmentation, guest hardening and database access controls.

Containers offer a smaller, faster deployment unit, but all containers on a host depend on the same kernel. A kernel vulnerability, excessive capabilities, privileged container, exposed container-daemon socket or unsafe host mount can weaken isolation. Use a non-root user, user namespaces/rootless mode where practical, drop capabilities, apply seccomp/AppArmor, limit devices and mounts, keep the host kernel/runtime patched and restrict who can control the container daemon. Docker's security guidance identifies namespaces, cgroups, daemon attack surface and kernel hardening as key considerations. [1]

4. Complexity Analysis and Final Evaluation

For N identical service instances, starting each instance is a constant operation relative to N, while starting a fleet sequentially is O(N). The container fleet and VM fleet therefore both have O(N) total instance-start work, but the per-instance time constant is usually much smaller for containers because no full guest OS boot is required. Parallel orchestration reduces wall-clock time but is bounded by available host capacity and startup concurrency.

A simplified memory model for containers is $O(M_{host} + N \times (M_{app} + M_{c}))$, because the host kernel is shared. For VMs it is $O(M_{host} + N \times (M_{guestOS} + M_{app} + M_{vmm}))$, because every VM includes a guest OS and virtualisation overhead. Both grow linearly with the number of instances; the VM model has a larger per-instance constant. Shared immutable image layers also avoid storing identical base files separately, while persistent volumes add storage according to actual database data and backup retention.

Recommendation: run multiple stateless portal containers for rapid scale-out and replaceability. Keep the production database in a managed database service or dedicated VM for durable storage and stronger separation. Use containerised database deployment only when persistent storage, replication, backup, recovery and least-privilege controls are fully designed and tested. This hybrid design balances seconds-level application startup and memory efficiency with stronger protection for persistent student-result data.

5. Conclusion

Linux namespaces isolate what container processes can see, while cgroups account for and limit selected resources. Container images are built from reusable read-only layers with a separate writable layer per running container; persistent database data belongs in volumes or managed storage. Compared with VMs, containers normally start faster and use less memory but share the host kernel, while VMs consume more resources and start more slowly but provide stronger isolation. For the result portal, stateless web services are best suited to containers, while the database should use a managed service or dedicated VM unless a secure stateful-container design is properly operated.

6. Technical References

[1] Docker Documentation, *Docker Engine Security* - namespaces, cgroups, daemon attack surface and kernel hardening. https://docs.docker.com/engine/security/

[2] Linux Kernel Documentation, *Control Group v2* - hierarchical process organisation and resource controllers. https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html

[3] Docker Documentation, *Understanding the Image Layers* - immutable layers, shared layers and stacked filesystems. https://docs.docker.com/get-started/docker-concepts/building-images/understanding-image-layers/

[4] Docker Documentation, *Storage* - writable container layers, volumes and persistence. https://docs.docker.com/engine/storage/

[5] Microsoft Learn, *Containers versus Virtual Machines* - shared-kernel container model, full guest OS in VMs and isolation/resource comparison. https://learn.microsoft.com/en-us/virtualization/windowscontainers/about/containers-vs-vm

Source alignment: Assignment 5 - CO5, Part A - Design, Task A4 - Evaluate Containerisation, 4 Marks, page 45 of the supplied question paper. General assignment requirements appear on page 42.